Workload Corpus — Families + the 13 Dwarfs

Two orthogonal divisions: 80 workloads by signature family, and the Berkeley 13 dwarfs · June 2026

The corpus has TWO orthogonal divisions of the SAME workloads. (1) The behaviour FAMILIES, organised by MEMORY SIGNATURE (what the write-signal actually clusters), kept as finalised: IDLE -- near-zero writes (CPU is its warm/active boundary); MEM -- working-set writes (CACHE is a footprint/locality sub-family); IO -- page-cache + metadata writes (cold reads count here); THREAD -- shared-line + allocator writes; BULK-REWRITE / encryptor -- high-entropy full rewrites (the ransomware cluster); ENUMERATION / metadata -- scanner-like; STEALTH / trickle -- low-rate, high-intensity; APP; and MIXED. This is the 'which behaviour' division -- designed by signature, validated by cohesion. (2) This document is a SECOND, CROSS-CUTTING division by the Berkeley 13 dwarfs (Colella's seven, 2004, + Berkeley's six, A View from Berkeley, 2006) -- the 'which computation motif' division. Every workload keeps its family label AND gets a dwarf label where one applies; the two taxonomies coexist, they do not replace each other. A dwarf is an algorithmic method that captures a pattern of computation and communication -- largely a MEMORY-ACCESS pattern, which is what the host memory signal sees.

We filter every dwarf by WRITE-visibility, because the signal only sees pages that are written. Visible / Visible++ = write-heavy, structured (a real signal). Irregular = visible writes with irregular access. Partial = part visible. Quiet = read/compute-bound -> near-idle (a CONTROL, the 'is this motif invisible to host introspection?' null). Threats are labelled motifs, not a separate family: ransomware = Combinational Logic (encryption), scanner = Graph Traversal / FSM.

Rules: a workload that already exists in another family is POINTED to (status 'exists', with its family), never duplicated. Each dwarf targets 4-5 distinct workloads. v1 pre-fills only the existing pointers and leaves the gaps; the new workloads are chosen together, dwarf by dwarf, in iterations (edit the WORKLOADS lists in the generator and re-run).

Part 1 — First division: behaviour families (80 workloads)

Every built workload by memory-signature family. The Dwarf column cross-references Part 2 (-- = access / IO / concurrency primitive, no motif).

S1 — IDLE (+ CPU boundary) (near-zero writes)

The no-write floor. CPU-bound workloads sit here as the warm/active boundary (pure compute is near-invisible to a write signal); matrix_mult drifts toward MEM via its output writes.

Workload	Status	Dwarf (Part 2)	Mechanism / note
run_idle	exists	--	sleep baseline
idle_long_baseline	exists	--	long uncontaminated idle (optional cache-drop)
idle_post_workload_recovery	exists	--	idle after a prior workload (writeback decay)
cpu_hash_loop_v2	exists	Combinational Logic	register-resident hash; near-idle (CPU boundary)
cpu_branch_random_v2	exists	Finite State Machines	random branches; near-idle (CPU boundary)
cpu_matrix_mult_v2	exists	Dense Linear Algebra	matmul; writes output C -> drifts to MEM
kernel_spmv_v2	under-testing	Sparse Linear Algebra	SpMV quiet control: gather-dominated, read-only structure -> near-idle (kernel/D2_quiet_sparse_linear_algebra/kernel_spmv_v2.c)

S2 — MEM (+ CACHE sub-family) (working-set writes)

Anonymous working-set writes. CACHE workloads are a footprint/locality sub-family (small = quiet, large = loud). These are access primitives, not computational motifs (no dwarf).

Workload	Status	Dwarf (Part 2)	Mechanism / note
mem_workingset_sweep_v2	exists	--	page-strided writes, footprint sweep
mem_writemag_sweep_v2	exists	--	per-page write-magnitude sweep
mem_rmw_intensity_v2	exists	--	read-modify-write intensity
mem_pagefault_density_v2	exists	--	fault vs steady-state touch
mem_mmap_traversal_v2	exists	--	mmap file traversal
mem_random_write_pages_v2	exists	--	random page writes
mem_stride_sweep_large_v2	exists	--	large-buffer stride sweep
cache_hot_loop_v2	exists	--	L1-resident RMW (CACHE sub; quiet)
cache_cold_scan_v2	exists	--	> LLC linear scan (CACHE sub)
cache_stride_sweep_v2	exists	--	> LLC stride (CACHE sub)

S3 — IO (page-cache + metadata writes)

Writes through the page cache + file metadata. Cold reads (cache fills) count here; io_read_cache_hit is the weak (near-idle) member.

Workload	Status	Dwarf (Part 2)	Mechanism / note
io_read_cache_hit_v2	exists	--	cache-hot reads; near-idle (weak member)
io_direct_write_like_v2	exists	--	O_DIRECT writes, bypass page cache

S4 — THREAD (shared-line + allocator writes)

Concurrency write patterns: shared cache lines, futex/scheduler, allocator churn. Contention types are quiet in APF (visible via temporal/revisit features).

Workload	Status	Dwarf (Part 2)	Mechanism / note
thread_lock_contention_v2	exists	--	mutex + shared cacheline
thread_producer_consumer_v2	exists	--	ring buffer + condvar
thread_parallel_alloc_v2	exists	--	concurrent malloc/free churn

S5 — BULK-REWRITE / encryptor (high-entropy full rewrites)

The threat-shaped cluster: discover -> read -> transform -> write -> rename over many files, high-entropy output. Dwarf: Combinational Logic (encryption).

Workload	Status	Dwarf (Part 2)	Mechanism / note
sandbox_ransom_seq	exists	Combinational Logic	sequential 5-phase per file
sandbox_ransom_batched	exists	Combinational Logic	batched phases (all-at-once)
sandbox_ransom_slowburn	exists	Combinational Logic	paced: one file / interval
sandbox_ransom_selective	exists	Combinational Logic	selective subset (.dat only)

S6 — ENUMERATION / metadata (stat-heavy, few writes)

Directory / metadata enumeration (stat, readdir), little content write. Dwarf: Graph Traversal.

Workload	Status	Dwarf (Part 2)	Mechanism / note
sandbox_scanner_metadata	exists	Graph Traversal	directory enumeration via stat

S7 — STEALTH / trickle (low-rate, high-intensity writes)

Low page-count, high per-page intensity -- the low-APF / high-Hamming probe. A modulation of a write motif, not a dwarf of its own.

Workload	Status	Dwarf (Part 2)	Mechanism / note
sandbox_stealth_microwrite	exists	--	single-page micro-rewrites
sandbox_stealth_scattered	exists	--	scattered page rewrites
sandbox_stealth_paced	exists	--	time-jittered single-page writes

S8 — APP (real-application write mixes)

Whole real applications (not primitives). Each also carries a dwarf label: compression = Combinational Logic, parsing = FSM, DB = MapReduce.

Workload	Status	Dwarf (Part 2)	Mechanism / note
app_hashtable_intensive_v2	exists	Combinational Logic	open-addressing hash build/probe
app_sqlite_oltp_v2	exists	MapReduce / Monte Carlo	OLTP WAL append + checkpoint
app_sqlite_analytical_v2	exists	MapReduce / Monte Carlo	analytical aggregation / scan
app_compress_gzip_v2	exists	Combinational Logic	gzip compress (LZ77 + Huffman)
app_decompress_gzip_v2	exists	Combinational Logic	gzip decompress
app_json_parse_v2	exists	Finite State Machines	streaming JSON parse

S9 — MIXED (blended write patterns)

Deliberate superpositions of other families; visible because their mem/io parts write. A blend by construction.

Workload	Status	Dwarf (Part 2)	Mechanism / note
mixed_mem_io_v2	exists	--	concurrent mem writes + file IO
mixed_cpu_mem_v2	exists	--	compute + mem pressure
mixed_cpu_io_v2	exists	--	cpu loop + file IO

S10 — KERNEL (compute motifs) (structured-compute writes)

Structured-compute writers -- the Berkeley dwarf kernels. Regular / periodic write patterns, distinct from MEM's amorphous sweeps. Each carries its dwarf label on axis 2. Built one at a time (see Status); the first is the D5 stencil.

Workload	Status	Dwarf (Part 2)	Mechanism / note
kernel_stencil_jacobi_v2	under-testing	Structured Grids	2D 5-point Jacobi; full-grid rewrite, double-buffer (kernel/D5_visible_structured_grids/kernel_stencil_jacobi_v2.c)

Workload	Status	Dwarf (Part 2)	Mechanism / note
kernel_stencil_seidel_v2	under-testing	Structured Grids	Gauss-Seidel red-black, in-place; checkerboard writes (kernel/D5_visible_structured_grids/kernel_stencil_seidel_v2.c)
kernel_multigrid_v2	under-testing	Structured Grids	geometric multigrid V-cycle; multi-scale, time-varying footprint (kernel/D5_visible_structured_grids/kernel_multigrid_v2.c)
kernel_fft_v2	under-testing	Spectral Methods	in-place radix-2 FFT; stage-varying-stride butterfly + bit-reversal scatter (kernel/D3_visible_spectral_methods/kernel_fft_v2.c)
kernel_gemm_v2	under-testing	Dense Linear Algebra	blocked dense matmul C=A*B; large output-C rewrite (kernel/D1_visible_dense_linear_algebra/kernel_gemm_v2.c)
kernel_nbody_v2	under-testing	N-Body Methods	2D particle sim; four compact arrays rewritten per step, smooth evolution (kernel/D4_visible_nbody_methods/kernel_nbody_v2.c)
kernel_dp_v2	under-testing	Dynamic Programming	edit-distance DP table fill; row-major wavefront (kernel/D10_visible_dynamic_programming/kernel_dp_v2.c)
kernel_hmm_v2	under-testing	Graphical Models	scaled HMM forward; probability-trellis column fill + dense transition matvec (kernel/D12_visible_graphical_models/kernel_hmm_v2.c)
kernel_lu_v2	under-testing	Dense Linear Algebra	in-place LU factorisation; shrinking trailing-submatrix front (kernel/D1_visible_dense_linear_algebra/kernel_lu_v2.c)
kernel_qr_v2	under-testing	Dense Linear Algebra	modified Gram-Schmidt / QR; growing orthogonalised-column front (kernel/D1_visible_dense_linear_algebra/kernel_qr_v2.c)
kernel_attention_v2	under-testing	Dense Linear Algebra	scaled dot-product attention (QK^T, row softmax, *V); transformer core (kernel/D1_visible_dense_linear_algebra/kernel_attention_v2.c)
kernel_conv_v2	under-testing	Dense Linear Algebra	2D convolution / CNN layer; overlapping-window MAC, feature-map rewrite (kernel/D1_visible_dense_linear_algebra/kernel_conv_v2.c)
kernel_ntt_v2	under-testing	Spectral Methods	multi-limb number-theoretic transform (modular butterfly); CKKS/lattice core, no crypto (kernel/D3_visible_spectral_methods/kernel_ntt_v2.c)
kernel_dct_v2	under-testing	Spectral Methods	blocked 8x8 2D DCT-II (JPEG transform); many small block rewrites (kernel/D3_visible_spectral_methods/kernel_dct_v2.c)
kernel_dwt_v2	under-testing	Spectral Methods	multi-level 2D Haar wavelet; shrinking multi-resolution pyramid (kernel/D3_visible_spectral_methods/kernel_dwt_v2.c)
kernel_fft2d_v2	under-testing	Spectral Methods	2D FFT (row FFTs, transpose, column FFTs); transpose scatter (kernel/D3_visible_spectral_methods/kernel_fft2d_v2.c)
kernel_barnes_hut_v2	under-testing	N-Body Methods	Barnes-Hut quadtree N-body; tree rebuilt each step + particle integrate (kernel/D4_visible_nbody_methods/kernel_barnes_hut_v2.c)
kernel_md_lj_v2	under-testing	N-Body Methods	Lennard-Jones molecular dynamics; cell list rebuilt each step + velocity-Verlet integrate (kernel/D4_visible_nbody_methods/kernel_md_lj_v2.c)
kernel_pic_v2	under-testing	N-Body Methods	electrostatic particle-in-cell; CIC scatter/gather + Jacobi Poisson solve on a grid (kernel/D4_visible_nbody_methods/kernel_pic_v2.c)

Workload	Status	Dwarf (Part 2)	Mechanism / note
kernel_fmm_v2	under-testing	N-Body Methods	single-level fast multipole; per-box complex expansion coefficients + far eval (kernel/D4_visible_nbody_methods/kernel_fmm_v2.c)
kernel_sph_v2	under-testing	N-Body Methods	smoothed-particle hydrodynamics; two-pass neighbour sum with per-particle density/pressure fields (kernel/D4_visible_nbody_methods/kernel_sph_v2.c)
kernel_lbm_v2	under-testing	Structured Grids	Lattice-Boltzmann D2Q9; 9 distribution arrays streamed + BGK collide each step (kernel/D5_visible_structured_grids/kernel_lbm_v2.c)
kernel_fdtd_v2	under-testing	Structured Grids	2D FDTD electromagnetics; coupled E/H field grids in Yee leapfrog (kernel/D5_visible_structured_grids/kernel_fdtd_v2.c)
kernel_floyd_v2	under-testing	Dynamic Programming	Floyd-Warshall all-pairs shortest paths; whole matrix relaxed n times (kernel/D10_visible_dynamic_programming/kernel_floyd_v2.c)
kernel_matrixchain_v2	under-testing	Dynamic Programming	matrix-chain optimal parenthesisation; anti-diagonal fill, O(n^3) (kernel/D10_visible_dynamic_programming/kernel_matrixchain_v2.c)
kernel_knapsack_v2	under-testing	Dynamic Programming	0/1 knapsack, space-optimised 1D rolling capacity array (kernel/D10_visible_dynamic_programming/kernel_knapsack_v2.c)
kernel_smithwaterman_v2	under-testing	Dynamic Programming	Smith-Waterman local alignment; wavefront fill + traceback (kernel/D10_visible_dynamic_programming/kernel_smithwaterman_v2.c)
kernel_beliefprop_v2	under-testing	Graphical Models	loopy sum-product belief propagation on a grid MRF; iterated message arrays (kernel/D12_visible_graphical_models/kernel_beliefprop_v2.c)
kernel_kalman_v2	under-testing	Graphical Models	ensemble of Kalman filters; small dense covariance matrices updated per step (kernel/D12_visible_graphical_models/kernel_kalman_v2.c)
kernel_gibbs_v2	under-testing	Graphical Models	Gibbs sampling on a Potts/Ising grid; stochastic per-cell resample sweep (kernel/D12_visible_graphical_models/kernel_gibbs_v2.c)
kernel_ldpc_v2	under-testing	Graphical Models	LDPC min-sum decoder; bipartite Tanner-graph message passing (kernel/D12_visible_graphical_models/kernel_ldpc_v2.c)
kernel_spmm_v2	under-testing	Sparse Linear Algebra	SpMM: sparse x dense -> dense output; the GNN aggregation kernel (kernel/D2_visible_sparse_linear_algebra/kernel_spmm_v2.c)
kernel_sparse_cholesky_v2	under-testing	Sparse Linear Algebra	banded sparse Cholesky; factor fills in within the band (kernel/D2_visible_sparse_linear_algebra/kernel_sparse_cholesky_v2.c)
kernel_spgemm_v2	under-testing	Sparse Linear Algebra	SpGEMM: sparse x sparse -> new sparse matrix, fill-in (kernel/D2_visible_sparse_linear_algebra/kernel_spgemm_v2.c)
kernel_sddmm_v2	under-testing	Sparse Linear Algebra	SDDMM: sampled dense-dense -> sparse output at mask positions (kernel/D2_visible_sparse_linear_algebra/kernel_sddmm_v2.c)
kernel_moe_dispatch_v2	under-testing	Sparse Linear Algebra	MoE dispatch: token-permutation scatter into expert buffers + combine (kernel/D2_visible_sparse_linear_algebra/kernel_moe_dispatch_v2.c)
kernel_fem_assembly_v2	under-testing	Unstructured Grids	FEM stiffness assembly: scatter-add element matrices into a global matrix (kernel/D6_visible_unstructured_grids/kernel_fem_assembly_v2.c)

Workload	Status	Dwarf (Part 2)	Mechanism / note
kernel_fem_matvec_v2	under-testing	Unstructured Grids	matrix-free FEM matvec: element gather-apply-scatter into a result vector (kernel/D6_visible_unstructured_grids/kernel_fem_matvec_v2.c)
kernel_dg_v2	under-testing	Unstructured Grids	discontinuous Galerkin step: per-element dense volume + face-flux coupling (kernel/D6_visible_unstructured_grids/kernel_dg_v2.c)
kernel_mesh_smooth_v2	under-testing	Unstructured Grids	unstructured Laplacian mesh smoothing over an adjacency list (kernel/D6_visible_unstructured_grids/kernel_mesh_smooth_v2.c)
kernel_unstructured_fv_v2	under-testing	Unstructured Grids	finite-volume: conservative face-flux scatter-add into cells (kernel/D6_visible_unstructured_grids/kernel_unstructured_fv_v2.c)

First division total: 80 workloads across 10 signature families.

Part 2 — Second division: the Berkeley 13 dwarfs

Coverage summary

Dwarf	Origin	Visibility	Maps to	Have	Target
D1 Dense Linear Algebra	Colella-7	Visible	KERNEL	**6**	4-5
D2 Sparse Linear Algebra	Colella-7	Visible	split: IDLE (spmv control) + KERNEL (writers)	**6**	4-5
D3 Spectral Methods	Colella-7	Visible++	KERNEL	**5**	4-5
D4 N-Body Methods	Colella-7	Visible	KERNEL	**6**	4-5
D5 Structured Grids	Colella-7	Visible++	KERNEL	**5**	4-5
D6 Unstructured Grids	Colella-7	Visible	KERNEL (irregular access)	**5**	4-5
D7 MapReduce / Monte Carlo	Colella-7	Partial	split	**1**	4-5
D8 Combinational Logic	Berkeley+6	Quiet / Visible	control OR threat-labeled	**4**	4-5
D9 Graph Traversal	Berkeley+6	Irregular	KERNEL-irregular / scanner	**1**	4-5
D10 Dynamic Programming	Berkeley+6	Visible	KERNEL	**5**	4-5
D11 Backtrack / Branch-and-Bound	Berkeley+6	Quiet	CPU/IDLE control	**0**	4-5
D12 Graphical Models	Berkeley+6	Visible	KERNEL	**5**	4-5
D13 Finite State Machines	Berkeley+6	Quiet	CPU/IDLE control / parser	**2**	4-5

D1 — Dense Linear Algebra [Visible]

Colella-7. Maps to: KERNEL. Example: GEMM, LU, QR, attention (transformers), convolution (CNNs), BLAS-3.

O(n^3) compute on O(n^2) data; regular, blocked access; rewrites the output / factorised matrix. Anchored on the kernels that dominate production: GEMM (every neural-net layer), LU/QR (linear solve, least-squares), plus attention + convolution (the transformer and CNN cores). Three distinct write fronts -- static full rewrite (GEMM), shrinking trailing front (LU), growing orthogonalised front (QR) -- with attention/conv added for real-world coverage (their write signature is ~GEMM).

Target 4-5 workloads — have 6.

Workload / Algorithm	Status	Mechanism / points-to	Memory signature	Used in (real world)
kernel_gemm_v2	under-testing	KERNEL family: blocked dense matmul (-> kernel/D1_visible_dense_linear_algebra/kernel_gemm_v2.c)	Visible (static full output-C rewrite)	Every neural-net layer; BLAS-3 (cuBLAS/MKL); scientific computing
cpu_matrix_mult_v2	exists	CPU family: naive square matmul, writes output C	Visible (regular; small-scale GEMM)	Small-scale GEMM control
kernel_lu_v2	under-testing	KERNEL family: in-place LU factorisation (-> kernel/D1_visible_dense_linear_algebra/kernel_lu_v2.c)	Visible (shrinking trailing-submatrix front)	Linear solve: SPICE circuit sim, finite-element, optimisation (LAPACK dgetrf)
kernel_qr_v2	under-testing	KERNEL family: Gram-Schmidt / QR orthogonalisation (-> kernel/D1_visible_dense_linear_algebra/kernel_qr_v2.c)	Visible (growing orthogonalised-column front)	Least-squares regression; GMRES/Arnoldi eigensolvers
kernel_attention_v2	under-testing	KERNEL family: scaled dot-product attention QK^T/softmax/*V (-> kernel/D1_visible_dense_linear_algebra/kernel_attention_v2.c)	Visible (transformer core; ~GEMM + row-softmax)	Every transformer / LLM (the per-token core)
kernel_conv_v2	under-testing	KERNEL family: 2D convolution / CNN layer, sliding-window MAC (-> kernel/D1_visible_dense_linear_algebra/kernel_conv_v2.c)	Visible (overlapping-window feature-map rewrite)	CNNs: image & video vision models
Cholesky factorisation	candidate	KERNEL (candidate): in-place symmetric factorisation, triangular shrinking front	Visible (LU-like triangular front)	Kalman filters, finance covariance, normal-equation least-squares
Triangular solve (TRSM)	candidate	KERNEL (candidate): forward/back substitution, column wavefront	Visible (column-wise solve front)	The solve step inside LU/Cholesky; preconditioners

D2 — Sparse Linear Algebra [Visible]

Colella-7. Maps to: split: IDLE (spmv control) + KERNEL (writers). Example: SpMV (quiet control), SpMM, sparse Cholesky, SpGEMM, SDDMM, MoE dispatch.

Sparse linear algebra is QUIET only when the OUTPUT is small: SpMV reads a big sparse matrix by indirect gather but writes just a vector, so it is near-idle -- the classic 'important but invisible' case (kept as kernel_spmv_v2, the control). But sparse work that produces a LARGE output IS write-visible, and that is most of modern practice: SpMM (a dense output, the graph-neural-network aggregation kernel), sparse Cholesky (fill-in factors), SpGEMM (a new sparse matrix, algebraic multigrid), SDDMM (a sampled sparse output, graph attention), and MoE dispatch (a token-permutation scatter, modern LLMs). So D2 spans a quiet half and a visible half.

Target 4-5 workloads — have 6.

Workload / Algorithm	Status	Mechanism / points-to	Memory signature	Used in (real world)
kernel_spmv_v2	under-testing	IDLE family (QUIET control): CSR SpMV, gather-dominated, tiny vector write (-> kernel/D2_visible_sparse_linear_algebra/kernel_spmv_v2.c)	Quiet / near-idle (the invisible baseline)	Inner loop of CG/GMRES solvers; recommenders; graph-as-matrix
kernel_spmm_v2	under-testing	KERNEL family (VISIBLE): sparse x dense -> dense output (-> kernel/D2_visible_sparse_linear_algebra/kernel_spmm_v2.c)	Visible (large dense output rewritten each pass)	Graph neural networks (the aggregation kernel; DGL/PyG)
kernel_sparse_cholesky_v2	under-testing	KERNEL family (VISIBLE): banded sparse Cholesky, fill-in (-> kernel/D2_visible_sparse_linear_algebra/kernel_sparse_cholesky_v2.c)	Visible (factor fills in within the band; progressive write)	Direct solvers (CHOLMOD/MUMPS): FEM, circuits, optimisation

Workload / Algorithm	Status	Mechanism / points-to	Memory signature	Used in (real world)
kernel_spgemm_v2	under-testing	KERNEL family (VISIBLE): sparse x sparse -> new sparse matrix (-> kernel/D2_visible_sparse_linear_algebra/kernel_spgemm_v2.c)	Visible (writes a new sparse matrix with fill-in)	Algebraic multigrid setup; triangle counting; graph contraction
kernel_sddmm_v2	under-testing	KERNEL family (VISIBLE): sampled dense-dense -> sparse output (-> kernel/D2_visible_sparse_linear_algebra/kernel_sddmm_v2.c)	Visible (scattered writes at the sparse mask positions)	Graph-attention networks; recommender systems
kernel_moe_dispatch_v2	under-testing	KERNEL family (VISIBLE): MoE token dispatch/combine scatter-gather (-> kernel/D2_visible_sparse_linear_algebra/kernel_moe_dispatch_v2.c)	Visible (token-permutation scatter into expert buffers)	Mixture-of-Experts LLMs (Mixtral, DeepSeek); >60% of 2025-26 releases
PageRank	covered	covered by kernel_spmv_v2 (repeated SpMV; same quiet gather + small vector write)	same quiet gather	Google web ranking; centrality; link analysis
Conjugate Gradient (CG)	covered	covered by kernel_spmv_v2 (SpMV-bound iteration, small vector writes)	same quiet gather	FEM/CFD SPD iterative solver
Sparse triangular solve (SpTRSV)	covered	covered by kernel_spmv_v2 (dependency-ordered sparse solve, tiny writes)	same quiet gather	ILU preconditioners

D3 — Spectral Methods [Visible++]

Colella-7. Maps to: KERNEL. Example: FFT, NTT, DCT, DWT, 2D-FFT (covers DTFT, cepstrum, MFCC, STFT, wavelet scattering).

Butterfly / bit-reversed, strided, multi-pass; in-place rewrite of the whole array. FIVE distinct write-patterns cover the dwarf: 1D butterfly (FFT), multi-stream butterfly (NTT), blocked (DCT), halving pyramid (DWT), transpose + two-direction (2D-FFT). Famous transforms that reduce to these -- DTFT, cepstrum, MFCC, STFT, wavelet scattering, FFT-convolution -- are marked 'covered' and point to the built test whose signature already captures them (building them would be pseudoreplication).

Target 4-5 workloads — have 5.

Workload / Algorithm	Status	Mechanism / points-to	Memory signature	Used in (real world)
kernel_fft_v2	under-testing	KERNEL family: in-place radix-2 FFT (-> kernel/D3_visible_spectral_methods/kernel_fft_v2.c)	Visible++ (1D butterfly; bit-reversal scatter)	All DSP: audio, radio/5G, MRI, image filtering
kernel_ntt_v2	under-testing	KERNEL family: RNS multi-limb number-theoretic transform, CKKS/lattice core (-> kernel/D3_visible_spectral_methods/kernel_ntt_v2.c)	Visible++ (multi-stream modular butterfly; integer content)	Lattice crypto / CKKS homomorphic encryption; big-integer multiply
kernel_dct_v2	under-testing	KERNEL family: blocked 8x8 discrete cosine transform (-> kernel/D3_visible_spectral_methods/kernel_dct_v2.c)	Visible (many small blocks; real content)	JPEG / MPEG / H.264 image & video compression
kernel_dwt_v2	under-testing	KERNEL family: discrete wavelet transform, filter+downsample pyramid (-> kernel/D3_visible_spectral_methods/kernel_dwt_v2.c)	Visible (halving multi-resolution pyramid)	JPEG2000, denoising, audio/image compression
kernel_fft2d_v2	under-testing	KERNEL family: 2D FFT (row FFTs, transpose, column FFTs) (-> kernel/D3_visible_spectral_methods/kernel_fft2d_v2.c)	Visible++ (transpose scatter + two-direction passes)	Image/optics spectral filtering, turbulence DNS, crystallography

Workload / Algorithm	Status	Mechanism / points-to	Memory signature	Used in (real world)
DTFT / direct DFT	covered	covered by kernel_fft_v2 (computable DTFT = DFT = FFT; naive DFT = dense matvec)	same 1D-butterfly signature	Frequency analysis (textbook form of the FFT)
Cepstrum	covered	covered by kernel_fft_v2 (FFT -> log|.| -> inverse FFT)	two butterfly passes + pointwise	Pitch detection, echo / speaker analysis
MFCC	covered	covered by kernel_fft_v2 + kernel_dct_v2 (FFT -> mel filterbank -> log -> DCT)	butterfly + blocked cosine	The classic speech-recognition audio feature
STFT / spectrogram	covered	covered by kernel_fft_v2 (sliding windowed FFTs filling a spectrogram)	repeated 1D butterfly (sliding window)	Every audio ML front-end; speech, music
Wavelet scattering (WST)	covered	covered by kernel_dwt_v2 (cascade of wavelet transforms + modulus)	repeated pyramid	Audio / image classification features
FFT convolution	covered	covered by kernel_fft_v2 (forward FFT, pointwise multiply, inverse FFT)	two butterfly passes + pointwise	Large-kernel convolution; polynomial / big-integer multiply

D4 — N-Body Methods [Visible]

Colella-7. Maps to: KERNEL. Example: gravity nbody, Barnes-Hut, molecular dynamics, particle-in-cell, FMM, SPH.

N particles interacting pairwise; rewrites compact particle arrays (position/velocity) smoothly each timestep. Variants differ mostly in how they READ to compute forces (all-pairs, tree, neighbour lists) -- invisible to the write-signal -- so they are distinguished by the EXTRA structure they WRITE: Barnes-Hut a tree, FMM expansion arrays, MD cell/neighbour lists, PIC a grid (scatter-deposit). All-pairs adds no new write, so it is 'covered' by the baseline nbody.

Target 4-5 workloads — have 6.

Workload / Algorithm	Status	Mechanism / points-to	Memory signature	Used in (real world)
kernel_nbody_v2	under-testing	KERNEL family: 2D K-sampled gravity (-> kernel/D4_visible_nbody_methods/kernel_nbody_v2.c)	Visible (four compact arrays rewritten per step, smooth)	2D gravity model; astrophysics, particle effects
kernel_barnes_hut_v2	under-testing	KERNEL family: quadtree build + traversal, O(n log n) (-> kernel/D4_visible_nbody_methods/kernel_barnes_hut_v2.c)	Visible/irregular (tree nodes rebuilt each step + particle rewrite)	Cosmology (GADGET), galaxy formation
kernel_md_lj_v2	under-testing	KERNEL family: Lennard-Jones MD with cell lists (-> kernel/D4_visible_nbody_methods/kernel_md_lj_v2.c)	Visible (periodic cell-list rebuild + position/velocity rewrite)	GROMACS / NAMD / AMBER: drug discovery, protein folding, materials
kernel_pic_v2	under-testing	KERNEL family: particle-in-cell (scatter to grid, field solve, gather) (-> kernel/D4_visible_nbody_methods/kernel_pic_v2.c)	Visible (particle->grid scatter-deposit + grid rewrite)	Plasma physics, accelerators, semiconductor device sim
kernel_fmm_v2	under-testing	KERNEL family: fast multipole (multipole/local expansion arrays on a tree) (-> kernel/D4_visible_nbody_methods/kernel_fmm_v2.c)	Visible (expansion-coefficient arrays + particle rewrite)	Electrostatics, acoustics, fast O(n) far-field
kernel_sph_v2	under-testing	KERNEL family: smoothed-particle hydrodynamics (density/pressure fields) (-> kernel/D4_visible_nbody_methods/kernel_sph_v2.c)	Visible (extra per-particle fields + particle rewrite)	Fluid sim, film VFX (water/lava), astrophysics
All-pairs direct N-body	covered	covered by kernel_nbody_v2 (same particle-array writes; differs only in the force READS)	same smooth particle rewrite	Exact small-N molecular dynamics; reference force

D5 — Structured Grids [Visible++]

Colella-7. Maps to: KERNEL. Example: stencil, Jacobi/Gauss-Seidel, multigrid, Lattice-Boltzmann, FDTD.

Regular neighbour sweep over a grid, iterative; rewrites the grid each iteration. Covered by 3 genuinely-distinct members (double-buffer / in-place / multigrid); further stencil variants (9-point, 3D, separable) differ only in reads/content -> signal-redundant. LBM/FDTD listed as distinct candidates (multiple field arrays, leapfrog).

Target 4-5 workloads — have 5.

Workload / Algorithm	Status	Mechanism / points-to	Memory signature	Used in (real world)
kernel_stencil_jacobi_v2	under-testing	KERNEL family: 2D 5-point Jacobi, double-buffer (-> kernel/D5_visible_structured_grids/kernel_stencil_jacobi_v2.c)	Visible++ (periodic full rewrite, ~2x footprint)	Finite-difference PDE: heat / Poisson / diffusion solvers
kernel_stencil_seidel_v2	under-testing	KERNEL family: Gauss-Seidel red-black, in-place (-> kernel/D5_visible_structured_grids/kernel_stencil_seidel_v2.c)	Visible++ (checkerboard in-place, ~1x footprint)	PDE relaxation; smoother inside multigrid
kernel_multigrid_v2	under-testing	KERNEL family: multigrid V-cycle (-> kernel/D5_visible_structured_grids/kernel_multigrid_v2.c)	Visible++ (multi-scale, time-varying footprint)	The optimal PDE solver; CFD, electrostatics
kernel_lbm_v2	under-testing	KERNEL family: Lattice-Boltzmann D2Q9, stream + collide over 9 distribution arrays (-> kernel/D5_visible_structured_grids/kernel_lbm_v2.c)	Visible++ (nine distribution arrays streamed each step)	CFD: porous media, aerodynamics (OpenLB / Palabos)
kernel_fdtd_v2	under-testing	KERNEL family: 2D TM FDTD, leapfrog of coupled E/H field grids (-> kernel/D5_visible_structured_grids/kernel_fdtd_v2.c)	Visible++ (two coupled field grids, E<->H leapfrog)	Antenna / radar / photonics simulation (Meep)

D6 — Unstructured Grids [Visible]

Colella-7. Maps to: KERNEL (irregular access). Example: FEM assembly, matrix-free FEM matvec, discontinuous Galerkin, mesh smoothing, unstructured finite-volume.

The irregular cousin of D5: PDE computation on unstructured meshes, reaching neighbours through explicit connectivity / adjacency lists (indirect gather). It writes real mesh/matrix arrays, so it is visible -- but the honest nuance is that the irregular ACCESS is mostly reads (invisible), so a plain unstructured relaxation writes the same footprint as a structured stencil. The genuinely distinct write is the SCATTER-ACCUMULATE that structured grids do not do: assembling a global matrix, or scatter-adding fluxes into cells. FEM matvec is the quieter member (a vector output, the matrix-free analog of SpMV).

Target 4-5 workloads — have 5.

Workload / Algorithm	Status	Mechanism / points-to	Memory signature	Used in (real world)
kernel_fem_assembly_v2	under-testing	KERNEL family: scatter-add element matrices into a global matrix (-> kernel/D6_visible_unstructured_grids/kernel_fem_assembly_v2.c)	Visible (indexed scatter-accumulate into a large matrix)	FEM assembly: ANSYS / Abaqus structural & crash; aerospace
kernel_fem_matvec_v2	under-testing	KERNEL family: matrix-free FEM matvec, element gather-apply-scatter (-> kernel/D6_visible_unstructured_grids/kernel_fem_matvec_v2.c)	Quieter (scatter-add into a result VECTOR; the matrix-free SpMV analog)	Large FEM/CFD iterative solvers (matrix-free)
kernel_dg_v2	under-testing	KERNEL family: discontinuous Galerkin step, per-element dense + face flux (-> kernel/D6_visible_unstructured_grids/kernel_dg_v2.c)	Visible (per-element dense blocks rewritten + flux coupling)	High-order CFD & seismic wave propagation

Workload / Algorithm	Status	Mechanism / points-to	Memory signature	Used in (real world)
kernel_mesh_smooth_v2	under-testing	KERNEL family: unstructured Laplacian mesh smoothing (-> kernel/D6_visible_unstructured_grids/kernel_mesh_smooth_v2.c)	Visible (node-array rewrite; write ~ D5 stencil, distinct in access)	Graphics mesh processing; remeshing
kernel_unstructured_fv_v2	under-testing	KERNEL family: finite-volume, conservative face-flux scatter-add into cells (-> kernel/D6_visible_unstructured_grids/kernel_unstructured_fv_v2.c)	Visible (face-list gather + conservative cell scatter-add)	OpenFOAM CFD; aerodynamics
Mesh partitioning (METIS)	candidate	KERNEL-irregular (candidate): graph coarsen / partition / refine	Irregular (graph rewrite; closer to D9)	Parallel FEM domain decomposition

D7 — MapReduce / Monte Carlo [Partial]

Colella-7. Maps to: split. Example: Monte-Carlo integration, option pricing, MCMC, word-count, path tracing.

Embarrassingly parallel map (writes intermediates) + reduce (small); or RNG accumulate. sqlite_analytical is a loose proxy; the real members are Monte-Carlo (quiet RNG-accumulate) and histogram/word-count (visible scatter-write map).

Target 4-5 workloads — have 1.

Workload / Algorithm	Status	Mechanism / points-to	Memory signature	Used in (real world)
app_sqlite_analytical_v2	exists	APP family (loose): read-heavy aggregation/scan over a DB	Quiet-ish (read-dominated)	Analytical SQL scan / aggregate (OLAP)
Monte-Carlo integration	candidate	IDLE (candidate): RNG sample + running accumulate	Quiet (register-resident accumulate)	Physics, finance, Bayesian; high-dimensional integrals
Monte-Carlo option pricing	candidate	IDLE (candidate): simulate many price paths, average payoff	Quiet (RNG + small accumulators)	Quant finance: derivatives, VaR, risk
MCMC (Metropolis-Hastings)	candidate	IDLE (candidate): proposal + accept/reject random walk	Quiet (small state rewrite)	Bayesian inference (Stan / PyMC), statistical physics
Histogram / word-count	candidate	MEM (candidate): scatter increments into bins / a hash map	Visible-ish (scattered bin writes)	MapReduce / Spark ETL; analytics
Path tracing	candidate	MEM (candidate): trace random rays, accumulate into an image buffer	Visible (image-buffer accumulation)	Film & game rendering (RenderMan, Blender Cycles)

D8 — Combinational Logic [Quiet / Visible]

Berkeley+6. Maps to: control OR threat-labeled. Example: compression, SHA-256, AES, CRC32, hashing.

Simple bit-level ops over large data. CRC/hash = quiet; ENCRYPTION = visible high-entropy rewrite.

Target 4-5 workloads — have 4.

Workload / Algorithm	Status	Mechanism / points-to	Memory signature	Used in (real world)
app_compress_gzip_v2	exists	APP family: LZ77+Huffman stream compress, writes output	Visible (entropy-changing)	Web (HTTP gzip), storage, backups (DEFLATE)
app_decompress_gzip_v2	exists	APP family: inverse, small read -> large write	Visible	Web / storage decompression
cpu_hash_loop_v2	exists	CPU family: FNV hash over a stream	Quiet (register-resident)	Hash tables, checksums
sandbox_ransom_* (4 variants)	exists	SECURITY family (THREAT-labeled): discover->read->XOR->write->rename	Visible++ (high-entropy rewrite)	Benign XOR; encryption-shaped rewrite control
SHA-256	candidate	CPU/IDLE (candidate): streaming compression function, tiny digest	Quiet (stream read, 32-byte write)	git, blockchain, TLS certificates, dedup, integrity

Workload / Algorithm	Status	Mechanism / points-to	Memory signature	Used in (real world)
AES block cipher	covered	covered by sandbox_ransom_* (same high-entropy full-rewrite signature)	Visible (high-entropy output rewrite)	HTTPS/TLS, disk encryption (BitLocker / FileVault), VPN
CRC32	candidate	CPU/IDLE (candidate): table-driven rolling checksum	Quiet (register-resident)	Ethernet, ZIP, storage error detection

D9 — Graph Traversal [Irregular]

Berkeley+6. Maps to: KERNEL-irregular / scanner. Example: BFS, DFS, Dijkstra/A, connected components.*

Visit objects via indirect lookups, little compute; writes visited/frontier/distance arrays. scanner_metadata is a directory-walk proxy; the real member is BFS (Graph500). Heavily used, but quiet (gather-dominated, small frontier/visited writes).

Target 4-5 workloads — have 1.

Workload / Algorithm	Status	Mechanism / points-to	Memory signature	Used in (real world)
sandbox_scanner_metadata	exists	SECURITY family (loose): directory enumeration via stat	Quiet / metadata	Directory-walk proxy
Breadth-first search (BFS)	candidate	KERNEL-irregular (candidate): frontier expand + visited-bitmap churn	Quiet/irregular (frontier + visited writes)	Graph500; shortest unweighted path; GC mark; social graph
Depth-first search (DFS)	candidate	KERNEL-irregular (candidate): explicit stack, visited marks	Quiet (stack + visited writes)	Topological sort, cycle detection, package resolvers (npm/cargo)
Dijkstra / A* shortest path	candidate	KERNEL-irregular (candidate): priority-queue relax, distance array	Quiet (heap + distance writes)	GPS routing, network routing, game pathfinding
Connected components	candidate	KERNEL-irregular (candidate): union-find or label propagation	Quiet (label array writes)	Clustering, image segmentation, fraud-ring detection

D10 — Dynamic Programming [Visible]

Berkeley+6. Maps to: KERNEL. Example: edit distance, Smith-Waterman, Viterbi, Floyd-Warshall, knapsack.

Fill a 1D/2D table, each cell from neighbours; regular monotone fill front (wavefront).

Target 4-5 workloads — have 5.

Workload / Algorithm	Status	Mechanism / points-to	Memory signature	Used in (real world)
kernel_dp_v2	under-testing	KERNEL family: edit-distance table fill (-> kernel/D10_visible_dynamic_programming/kernel_dp_v2.c)	Visible (row-major wavefront; migrating band on a large table)	git diff, spell-check, fuzzy matching
kernel_floyd_v2	under-testing	KERNEL family: Floyd-Warshall all-pairs shortest paths (-> kernel/D10_visible_dynamic_programming/kernel_floyd_v2.c)	Visible (whole n*n matrix rewritten n times per solve)	All-pairs shortest path; routing; transitive closure
kernel_matrixchain_v2	under-testing	KERNEL family: matrix-chain optimal parenthesisation (-> kernel/D10_visible_dynamic_programming/kernel_matrixchain_v2.c)	Visible (anti-diagonal fill by chain length, O(n^3) inner)	Compiler / query-plan optimisation, NLP parsing
kernel_knapsack_v2	under-testing	KERNEL family: 0/1 knapsack, space-optimised 1D rolling array (-> kernel/D10_visible_dynamic_programming/kernel_knapsack_v2.c)	Visible (single capacity vector repainted in reverse per item)	Resource allocation, scheduling, finance

Workload / Algorithm	Status	Mechanism / points-to	Memory signature	Used in (real world)
kernel_smithwaterman_v2	under-testing	KERNEL family: Smith-Waterman local alignment, fill + traceback (-> kernel/D10_visible_dynamic_programming/kernel_smithwaterman_v2.c)	Visible (row-major wavefront + backward traceback path)	Genomics local alignment (BLAST family)
Needleman-Wunsch	covered	covered by kernel_dp_v2 (identical global-alignment row-major wavefront)	same row-major wavefront	Global DNA / protein sequence alignment
Viterbi decoding	covered	covered by kernel_hmm_v2 (same trellis column-fill, max-product instead of sum)	same column-fill front	Speech recognition, error-correction decode, POS tagging

D11 — Backtrack / Branch-and-Bound [Quiet]

Berkeley+6. Maps to: CPU/IDLE control. Example: N-queens, Sudoku, DPLL/CDCL SAT, branch-and-bound MILP, TSP.

Explore + prune a search tree; writes a small search stack / current solution. Real and heavily used (SAT, MILP) but quiet -- deep recursion over a tiny working set. No test built yet.

Target 4-5 workloads — have 0.

Workload / Algorithm	Status	Mechanism / points-to	Memory signature	Used in (real world)
N-queens	candidate	CPU/IDLE (candidate): place + backtrack over a board	Quiet (tiny board state)	Classic constraint-satisfaction benchmark
Sudoku solver	candidate	CPU/IDLE (candidate): constraint propagation + backtracking	Quiet (81-cell grid)	Constraint-propagation teaching / puzzle solvers
DPLL / CDCL SAT	candidate	CPU/IDLE (candidate): unit-propagate, decide, learn clauses, backtrack	Quiet (clause DB reads, small writes)	Hardware/chip verification, planning (MiniSat / Z3)
Branch-and-bound MILP	candidate	CPU/IDLE (candidate): LP-relaxation bound, branch, prune	Quiet (search tree, small writes)	Logistics, scheduling, optimisation (Gurobi / CPLEX)
TSP branch-and-bound	candidate	CPU/IDLE (candidate): tour bound + prune search	Quiet (path/stack writes)	Routing, VLSI, operations research

D12 — Graphical Models [Visible]

Berkeley+6. Maps to: KERNEL. Example: HMM, Kalman filter, LDPC belief propagation, loopy BP, Gibbs sampling.

Probability ops over a graph; writes belief/message tables (matrix-like).

Target 4-5 workloads — have 5.

Workload / Algorithm	Status	Mechanism / points-to	Memory signature	Used in (real world)
kernel_hmm_v2	under-testing	KERNEL family: scaled HMM forward, trellis fill (-> kernel/D12_visible_graphical_models/kernel_hmm_v2.c)	Visible (column-fill front + dense matvec; normalised-probability content)	Speech recognition, gene finding, time-series
kernel_beliefprop_v2	under-testing	KERNEL family: loopy sum-product belief propagation on a grid MRF (-> kernel/D12_visible_graphical_models/kernel_beliefprop_v2.c)	Visible (iterated message arrays per grid cell)	Stereo vision, image denoising, MRF / CRF
kernel_kalman_v2	under-testing	KERNEL family: ensemble of Kalman filters, dense covariance updates (-> kernel/D12_visible_graphical_models/kernel_kalman_v2.c)	Visible (many small d*d covariance matrices rewritten per step)	GPS/INS navigation, object tracking, sensor fusion

Workload / Algorithm	Status	Mechanism / points-to	Memory signature	Used in (real world)
kernel_gibbs_v2	under-testing	KERNEL family: Gibbs sampling on a Potts/Ising grid (-> kernel/D12_visible_graphical_models/kernel_gibbs_v2.c)	Visible (stochastic per-cell resample sweep of the grid)	Bayesian inference, topic models (LDA)
kernel_ldpc_v2	under-testing	KERNEL family: LDPC min-sum decoder, message passing on a Tanner graph (-> kernel/D12_visible_graphical_models/kernel_ldpc_v2.c)	Visible (bipartite edge-message arrays iterated)	5G / WiFi / SSD / satellite error correction

D13 — Finite State Machines [Quiet]

Berkeley+6. Maps to: CPU/IDLE control / parser. Example: JSON parse, regex/DFA, Aho-Corasick, HTTP parser, lexer.

Read a stream, transition through states via table lookup; tiny memory write. Ubiquitous in systems software (parsing, regex, protocol decode) but quiet -- stream read + branch, tiny state.

Target 4-5 workloads — have 2.

Workload / Algorithm	Status	Mechanism / points-to	Memory signature	Used in (real world)
app_json_parse_v2	exists	APP family: streaming JSON parse (the canonical FSM)	Quiet (read + branch)	Every REST / JSON API; config parsing
cpu_branch_random_v2	exists	CPU family (loose): data-dependent random branches	Quiet	Branch-predictor control
Regex / DFA matcher	candidate	CPU/IDLE (candidate): table-driven state transitions over a stream	Quiet (state-table lookups, tiny writes)	grep, input validation, log processing
Aho-Corasick	candidate	CPU/IDLE (candidate): multi-pattern automaton over a stream	Quiet (goto/fail table reads)	Antivirus / IDS multi-pattern scan (ClamAV / Snort)
HTTP / protocol parser	candidate	CPU/IDLE (candidate): byte-by-byte protocol state machine	Quiet (small parse-state writes)	Web servers (nginx), TCP/IP stacks, deep-packet inspection
Lexer / tokenizer	candidate	CPU/IDLE (candidate): character-class FSM emitting tokens	Quiet (token-buffer writes)	Every compiler / interpreter front-end

Sources

A View from Berkeley (tech report)

The 13 Motifs of Parallel Programming

Reprising the 13 Dwarfs of OpenCL (HPCwire)